

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – Test Case Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 1 – Test Case Design Assignment
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Jasim Asmaan A	Reg. No.:	192421060
Date:	23/08/2026	Duration:	60 minutes

Code of Conduct

I Jasim Asmaan A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Jasim Asmaan A

Annexure A – Test Case Design Assignment

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design and document comprehensive test cases** to verify the correctness, functionality, reliability, and usability of the proposed system.

Your test case design should include:

1. Identify the **functional and non-functional requirements** to be tested.
2. Identify the **test scenarios** for the assigned problem statement.
3. Prepare detailed **test cases** covering normal, boundary, invalid, and exceptional conditions.
4. Include **Test Case ID, Test Scenario, Test Steps, Test Data, Expected Result, Actual Result, and Pass/Fail Status**.
5. Apply appropriate **test design techniques** such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, or State Transition Testing wherever applicable.
6. Ensure that the test cases provide adequate **requirement and functional coverage** for the assigned system.

Deliverable: Submit a Test Case Design document containing the identified scenarios, test cases, test data, expected results, and test coverage based on the individual problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1
---------------------	---------------------	----------------	------------------------	----------------------

				Mark)
Requirement & Test Scenario Identification(15 %)	Clearly identifies all relevant functional/non-functional requirements and comprehensive test scenarios	Identifies most requirements and scenarios	Identifies basic requirements and scenarios	Requirements/sce narios are incomplete or unclear
Test Case Design & Completeness(25 %)	Comprehensive test cases covering normal, boundary, invalid, and exceptional conditions	Covers most important conditions with minor gaps	Covers basic conditions but misses several important cases	Test cases are very limited or incomplete
Test Data & Expected Results(20%)	Appropriate test data with precise, measurable expected results for every test case	Mostly appropriate test data and expected results	Some test data/results are unclear or incomplete	Test data and expected results are largely missing/incorrect
Application of Test Design Techniques(15%)	Correctly applies multiple techniques such as Equivalence Partitioning, Boundary Value Analysis, Decision Table, and State Transition	Correctly applies at least two relevant techniques	Applies one technique with limited justification	Techniques are not applied or incorrectly applied
Traceability & Test Coverage(15%)	Excellent mapping between requirements, scenarios, and test cases with strong coverage	Good traceability and coverage with minor gaps	Partial traceability and moderate coverage	Poor/no traceability and inadequate coverage
Documentation & Presentation(10 %)	Well-structured, clear, professional, consistent, and easy to understand	Well-organized with minor presentation issues	Adequately organized but has some clarity/formatting issues	Poorly organized, unclear, or incomplete documentation

Criteria	Mark
Requirement & Test Scenario Identification (15%)	/5
Test Case Design & Completeness (25%)	/4
Test Data & Expected Results (20%)	/4
Application of Test Design Techniques (15%)	/4
Traceability & Test Coverage (15%)	/4
Documentation & Presentation (10%)	/4
TOTAL	/25

Assessment Approach and Model-Answer Method

This document follows the supplied model-answer method: identify functional and non-functional requirements, define test scenarios, design detailed test cases, apply test-design techniques, map requirements to tests, and summarize coverage.

The same structure is adapted to The AI-Based Smart Disaster Alert and Emergency Response Coordination System.

Rubric Area	Excellent level Approach used
Requirement & Scenario Identification (15%)	Complete functional/non-functional requirements and comprehensive scenarios.
Test Data & Expected Results (20%)	Specific data and precise expected outcomes.
Test Design Techniques (15%)	Equivalence Partitioning, Boundary Value Analysis, Decision Table and State Transition.
Traceability & Coverage (15%)	Requirement-to-test mapping and coverage summary.
Documentation & Presentation (10%)	Consistent IDs, headings, tables and professional format.

1. Functional and Non-Functional Requirements

Functional Requirements

ID	Requirement
FR01	Authorized users can register and log into the emergency response system.
FR02	Users can provide and update their current location.
FR03	Disaster type, location, severity, date/time and alert status are displayed correctly.
FR04	Users can search/filter disaster alerts based on location and disaster type.
FR05	Valid disaster information can be submitted/recorded.
FR06	Disaster information and user inputs are validated.
FR07	The AI system analyses disaster-related data and identifies potential threats.
FR08	A disaster risk score/risk level is calculated.
FR09	High-risk disasters generate emergency alerts to affected users.
FR10	Authorized emergency personnel can view/update alert and response status.
FR11	Frontend communicates correctly with backend APIs.
FR12	Backend returns valid disaster, alert and emergency-response information.
FR13	Invalid/unavailable API requests are handled appropriately.
FR14	Disaster alerts, emergency actions and response history are maintained.
FR15	Dashboard presents disaster monitoring and emergency response information.

Non-Functional Requirements

ID	Requirement	Testing Focus
NFR01	Usability	Interface, alerts and messages should be easy to understand.
NFR02	Performance	Alerts and normal requests should respond within an acceptable time.
NFR03	Reliability	System should operate without unexpected crashes.
NFR04	Availability	Emergency services should remain accessible when required.
NFR05	Security	Unauthorized access and unsafe inputs should be handled safely.
NFR06	Maintainability	System modules should be understandable and testable.
NFR07	Scalability	System should support increasing users and disaster-alert volume.
NFR08	Compatibility	Application should work in supported browsers and devices.
NFR09	Portability	Containerized/deployed services should run consistently.
NFR10	Data Integrity	Disaster, location, alert and response records should not be corrupted or lost.

2. Test Scenarios

ID	Scenario	Objective
TS01	Application Access	Open emergency response application successfully.
TS02	Valid Login	Authenticate with valid credentials.
TS03	Invalid Login	Reject incorrect credentials safely.
TS04	Disaster Alert Display	Display disaster alert details correctly.
TS05	Alert Search/Filter	Find relevant alerts using search/filter.
TS06	Valid Disaster Information	Accept and store valid disaster information.
TS07	Invalid Disaster Input	Reject empty, malformed or invalid data.
TS08	Severity Boundary	Verify disaster severity boundary values.
TS09	AI Disaster Analysis	Return disaster risk score/risk level.
TS10	Risk Scoring	Return risk score/risk level.
TS11	Emergency Alert	Generate alert for high-risk disaster.
TS12	Emergency Response Management	View/update emergency response status.
TS13	Frontend–Backend API	Verify correct API communication.
TS14	Invalid API/Error Handling	Handle invalid endpoint/service errors safely.
TS15	System Deployment	Start services and verify communication.
TS16	Restart & Availability	Restart services and verify application availability.

3. Test Design Techniques

Equivalence Partitioning

Input Class	Example	Expected
Valid severity	Severity 3	Accept
Zero amount	Severity 1	Accept
Invalid low severity	Severity 0	Reject
Invalid high severity	Severity 6	Reject
Non-numeric severity	ABC	Reject
Empty severity	Blank	Reject

Boundary Value Analysis

Boundary	Example	Expected
Below minimum	Severity 0	Reject
Minimum boundary	Severity 1	Accept
Just above minimum	Severity 2	Accept
Normal	Severity 3	Accept
Maximum boundary	Severity 5	Accept
Above maximum	Severity 6	Reject

Decision Table

Disaster Data	Risk Level	User in Affected Area	Expected
Valid	Low/Medium	Yes	Display information/normal notification
Valid	High	Yes	Generate emergency alert
Valid	High	No	Record disaster and notify relevant authorities
Invalid	Any	Yes	Reject input
Valid	Any	No	Store information and do not send location-specific alert

State Transition Testing

Normal Disaster Flow:

Submitted→Validated→AI Analysed→Low/MediumRisk→ Information/Notification → Completed

High-Risk Disaster Flow:

Submitted→Validated→AI Analysed→High Risk→Alert Generated→Emergency Response→ Under Response→Resolved

Invalid Flow:

Submitted→Validation Failed→Rejected

4. Detailed Test Cases

TC01– Open Emergency Response Application

Field	Details
Test Case ID	TC01
Test Scenario	Open Emergency Response Application
Test Steps	Start services; open browser; enter application URL.
Test Data	http://localhost:5173
Expected Result	Login/home page loads successfully.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING – update after execution.

TC02–Valid User Login

Field	Details
Test Case ID	TC02
Test Scenario	Valid User Login
Test Steps	Open login; enter valid credentials; click Login.
Test Data	citizen / valid test password
Expected Result	User is authenticated and dashboard opens.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING – update after execution.

TC03–Invalid User Login

Field	Details
Test Case ID	TC03
Test Scenario	Invalid User Login
Test Steps	Enter incorrect credentials and submit.
Test Data	citizen/ wrong123
Expected Result	Login is rejected with safe error message.
Actual Result	To be verified during actual execution.
Pass/FailStatus	PENDING–update after execution.

TC04–DisplayTransaction

Field	Details
Test Case ID	TC04
Test Scenario	Display Disaster Alert
Test Steps	Login; open Alerts; select an alert record.
Test Data	ALERT1001
Expected Result	Disaster type, location, severity, date/time and alert status are displayed correctly.
Actual Result	To be verified during actual execution.
Pass/FailStatus	PENDING–update after execution.

TC05–Search Disaster Alert

Field	Details
Test Case ID	TC05
Test Scenario	Search Disaster Alert
Test Steps	Enter disaster location/type in search
Test Data	Chennai / Flood
Expected Result	Matching disaster alerts are displayed.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING–update after execution.

TC06–Valid Disaster Information

Field	Details
Test Case ID	TC06
Test Scenario	Valid Disaster Information
Test Steps	Enter valid disaster information and submit.
Test Data	Flood, Chennai, Severity 3
Expected Result	Disaster information is validated and stored successfully.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING–update after execution.

TC07–Empty Mandatory Field

Field	Details
Test Case ID	TC07
Test Scenario	Empty Mandatory Field
Test Steps	Leave disaster type/location blank and submit.
Test Data	Blank disaster type
Expected Result	Validation message is displayed; record is not created.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING–update after execution.

TC08–Minimum Severity Boundary

Field	Details
Test Case ID	TC08
Test Scenario	Minimum Severity Boundary
Test Steps	Enter minimum valid disaster severity and submit.
Test Data	Severity 1
Expected Result	Severity value is accepted and disaster information is processed
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING– update after execution.

TC09–Invalid Severity

Field	Details
Test Case ID	TC09
Test Scenario	Invalid Severity
Test Steps	Enter an invalid severity value and submit.
Test Data	Severity 6
Expected Result	Invalid severity is rejected with a validation message.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING– update after execution.

TC10–Non-Numeric Severity

Field	Details
Test Case ID	TC10
Test Scenario	Non-Numeric Severity
Test Steps	Enter letters in the severity field and submit.
Test Data	ABC
Expected Result	Input is rejected; no invalid record is created.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING – update after execution.

TC11– AI Disaster Risk Analysis

Field	Details
Test Case ID	TC11
Test Scenario	AI Disaster Risk Analysis
Test Steps	Submit valid disaster data; run AI
Test Data	Flood, Chennai, Severity 4, heavy rainfall indicators
Expected Result	AI service analyses the data and returns a disaster risk score/level.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING – update after execution.

TC12–High-Risk Emergency Alert

Field	Details
Test Case ID	TC12
Test Scenario	High-Risk Emergency Alert
Test Steps	Submit high-risk disaster data; inspect alerts.
Test Data	High-risk flood/disaster test data
Expected Result	Emergency alert is generated and linked to the affected location/disaster.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING – update after execution.

TC13–Frontend–BackendAPI

Field	Details
Test Case ID	TC13
Test Scenario	Frontend–Backend API
Test Steps	Request disaster-alert data from frontend.
Test Data	GET /api/alerts
Expected Result	Valid backend response is received and displayed correctly.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING– update after execution.

TC14–InvalidAPIRequest

Field	Details
Test Case ID	TC14
Test Scenario	Invalid API Request
Test Steps	Call an invalid endpoint.
Test Data	GET/api/invalid
Expected Result	Appropriate error response is returned; service does not crash.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING– update after execution.

TC15– System Deployment

Field	Details
Test Case ID	TC15
Test Scenario	System Deployment
Test Steps	Run Docker Compose; inspect application
Test Data	docker compose up -d
Expected Result	Required services start successfully and communicate with each other.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING – update after execution.

TC16– System Restart

Field	Details
Test Case ID	TC16
Test Scenario	System Restart
Test Steps	Stop and restart services; reopen application.
Test Data	docker compose down / up -d
Expected Result	Services restart successfully and application becomes available.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING– update after execution.

TC17–Browser Compatibility

Field	Details
Test Case ID	TC17
Test Scenario	Browser Compatibility
Test Steps	Open application in a supported browser; use dashboard and alerts.
Test Data	Chrome / supported browser
Expected Result	Core pages, alerts and dashboard operate correctly.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING – update after execution.

TC18–Unauthorized API Access

Field	Details
Test Case ID	TC18
Test Scenario	Unauthorized API Access
Test Steps	Call a protected emergency-response endpoint without authentication.
Test Data	Protected endpoint without token
Expected Result	Access is denied safely and sensitive information is not exposed.
Actual Result	To be verified during actual execution.
Pass/Fail Status	PENDING – update after execution.

5. Requirement Traceability

Requirement	Covered Area	Test Cases
FR01	Login/access	TC01–TC03
FR02–FR03	Location and alert display	TC04
FR04	Alert search/filter	TC05
FR05	Disaster information recording	TC06
FR06	Input validation	TC07–TC10
FR07–FR08	AI analysis/risk scoring	TC11
FR09	Emergency alert generation	TC12
FR10	Emergency response management	TC12; add response-status update test
FR11–FR12	Frontend/API	TC13
FR13	Error handling	TC03,TC07–TC10,TC14
FR14	Alert/response history	TC04,TC12; add persistence verification
FR15	Emergency dashboard	TC04, TC13
NFR03–NFR04	Reliability/availability	TC15–TC16
NFR05	Security	TC03,TC18
NFR08	Compatibility	TC17
NFR09	Portability	TC15–TC16
NFR02	Performance	Dedicated response-time /load test required

6. Test Coverage Summary

Metric	Result/ Plan
Total designed test cases	18
Normal/positive cases	TC01,TC02,TC04–TC06, TC08, TC11–TC13, TC15–TC17
Invalid/negative cases	TC03,TC07,TC09,TC10,TC14,TC18
Boundary tests	TC08–TC09
Equivalence Partitioning	TC07–TC10
Boundary Value Analysis	TC08–TC09
Decision Table	Disaster validity × risk × affected location
State Transition	Submitted → Validated → Analysed → Risk/Alert states
Functional coverage	Authentication, location, disaster analysis, alerts, emergency response, API, deployment
Non-functional coverage	Security, reliability, availability, compatibility, portability; performance needs dedicated execution

7. Evidence/Screenshots to Attach

- Login page / emergency response dashboard
- User location/update screen
- Disaster alert list and alert details
- AI-generated disaster risk score
- High-risk emergency alert
- Emergency response status/update
- Backend API response
- Docker Compose services running
- Test execution results

Use screenshots from the actual project execution. Do not claim an execution result unless it has been tested.

8. Conclusion

This test case design covers the major functional and non-functional requirements of The AI-Based Smart Disaster Alert and Emergency Response Coordination System. It includes normal, boundary, invalid and exceptional conditions and applies Equivalence Partitioning, Boundary Value Analysis, Decision Table and State Transition Testing.

The traceability matrix maps requirements to test cases and highlights remaining gaps. Before final submission, execute the application and replace the Actual Result and PENDING status fields with the observed results and attach the corresponding screenshots.